

net-03 - Openflow e múltiplas Flow Tables

Contexto [↗](#)

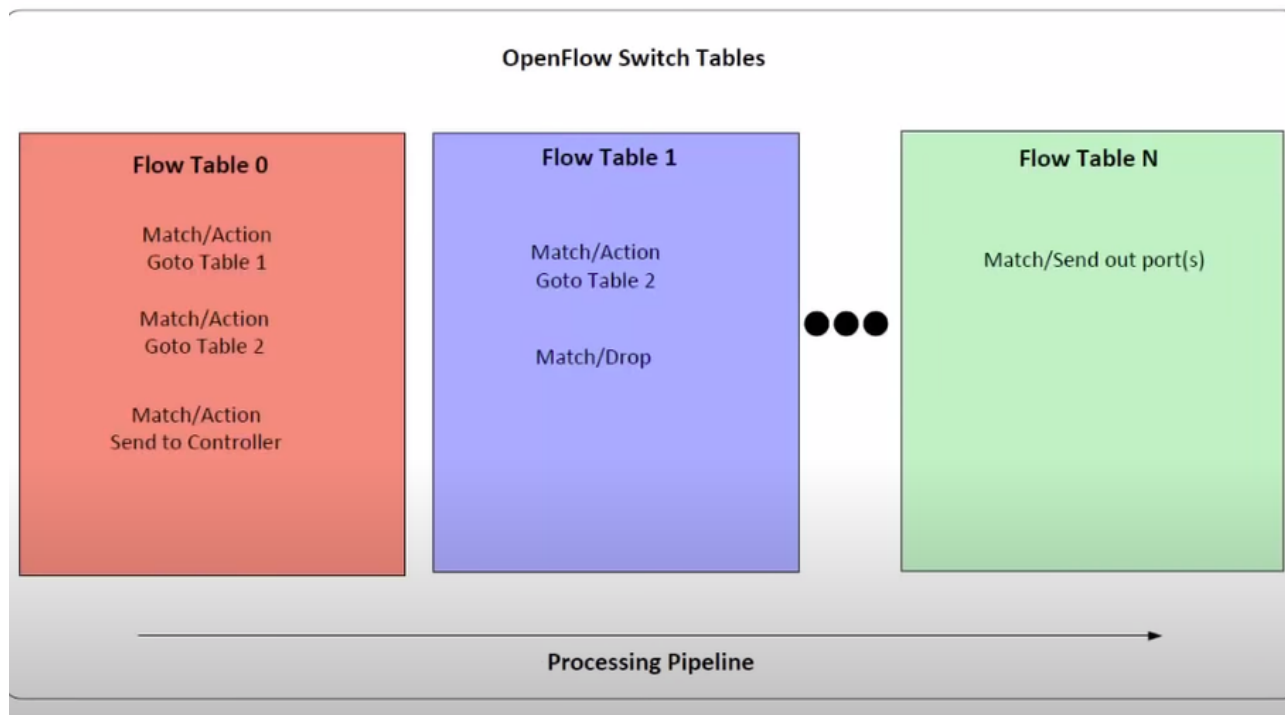
Neste treinamento iremos abordar:

- Openflow e uso de múltiplas Flow Tables
- Hands on - Manipulando o OVS com o Openflow usando múltiplas flow tables
- Hands on - Desafio

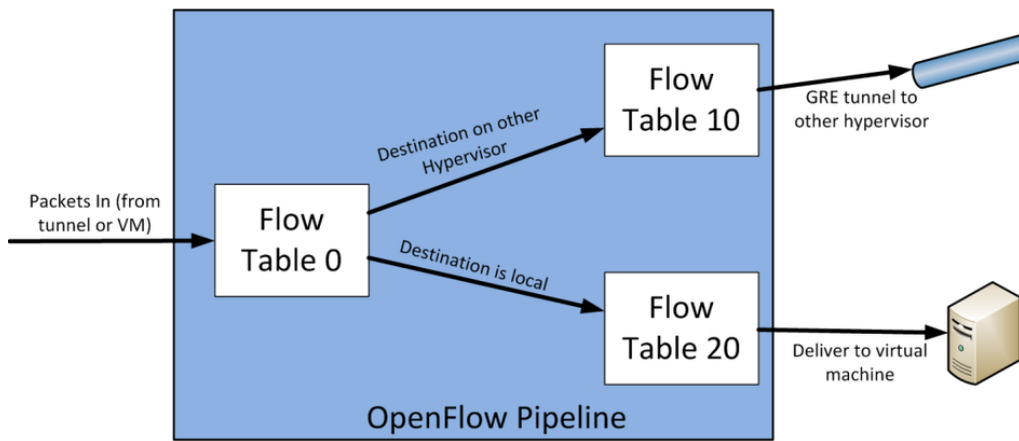
Openflow e o uso de múltiplas Flow Tables [↗](#)

Uma Flow table agrega um conjunto de flows que um pacote irá percorrer para ocorrer um **flow matching**.

Na versão 1.0 do Openflow, existia apenas uma flow table e conforme a evolução do protocolo foi ocorrendo, foram adicionadas novas flows tables para se otimizar o pipeline do Openflow. O **pipeline** é um termo muito comum que se refere a todo trajeto que o pacote percorre desde sua entrada na NIC até sua saída. Algumas flow tables possuem funções específicas, como realizar o lookup do pacote, executar alterações no header do pacote ou como executar o NAT por exemplo.

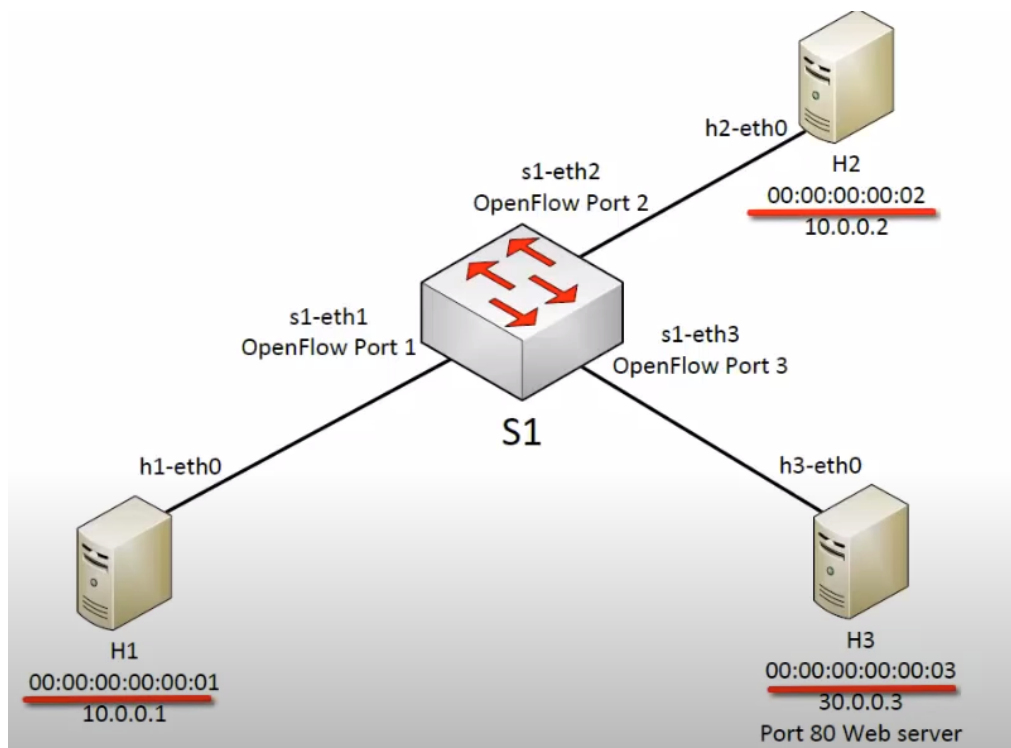


Todo o pacote entra na **Table 0** e dependendo da decisão que os flows dessa table tomam (baseado no header do pacote), o pacote pode ser enviado (**resubmitted**) via **action** para outra table para percorrer outro fluxo. O pipeline sempre é executado da esquerda para a direita, ou seja, o pacote que estiver na table 1 não poderá voltar para a table 0.



Dependendo do **packet lookup** executado em uma flow table, a action pode determinar que um pacote deva continuar o pipeline localmente em outra table até sair por uma porta de uma VM ou dependendo da action ser enviado para um outro host na rede que possua um switch Openflow.

Vamos estudar como funcionaria o pipeline com um exemplo hipotético, digamos que temos a seguinte topologia com os hosts H1, H2 na mesma subnet(10.0.0.0/24) e o H3 em uma segunda subnet(30.0.0.0/24) e o switch virtual S1:



Analisando os flows de cada **flow table** no próximo diagrama, podemos determinar:

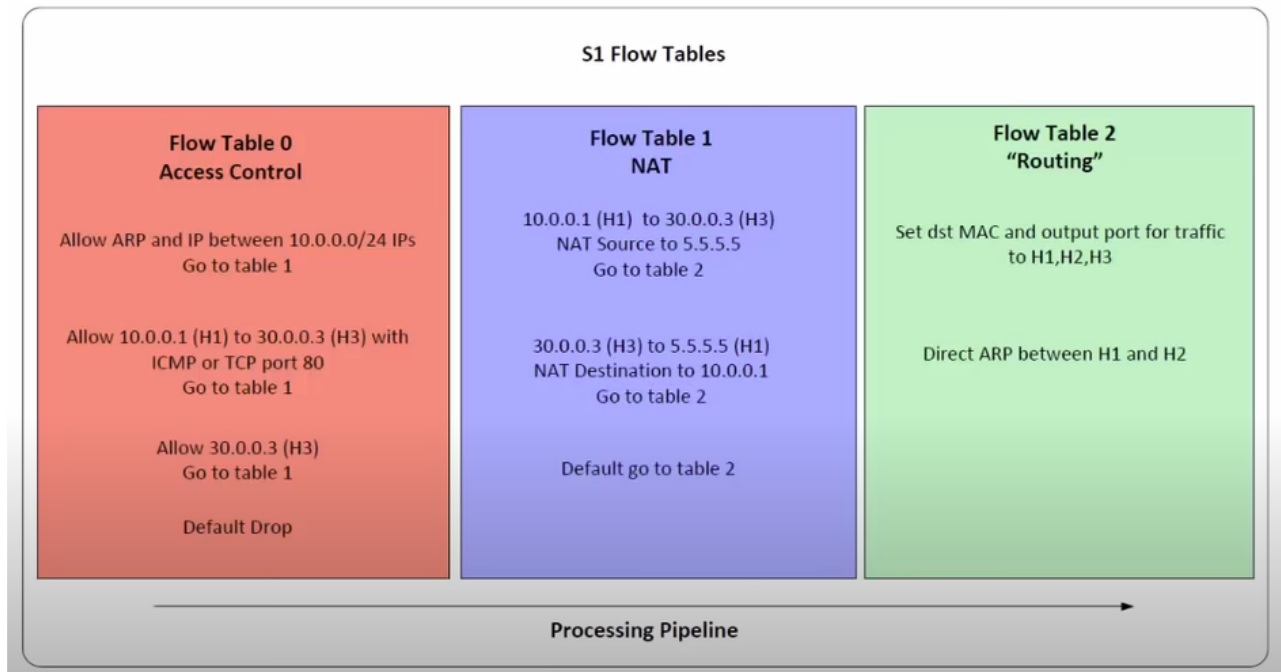
A **flow table 0** irá funcionar com um **classifier**, ou seja, irá classificar o pacote de acordo com seu header e irá encaminhar ou bloquear o fluxo daquele pacote.

O primeiro flow que existe é de permitir a resolução ARP entre qualquer IP que esteja na subnet 10.0.0.0/24, onde o tráfego de ARP é enviado para a **flow table 1** como **action**.

O segundo flow na **flow table 0**, permite o tráfego de rede a partir do host 10.0.0.1(H1) de ICMP e L4 referente a porta 80 para o destino 30.0.0.3(H3) e como **action** o pacote é enviado para a **flow table 1**.

O terceiro flow na **flow table 0**, permite o tráfego de rede do host 30.0.0.3(H3) e os pacotes são enviados para a **flow table 1** como action.

O quarto flow na **flow table 0**, irá bloquear(drop) o tráfego de rede para qualquer pacote que não tenha **flow matching** com os flows anteriores.



Seguindo a pipeline, a **flow table 1** irá executar o NAT dos pacotes que foram encaminhados a partir da **flow table 0**.

O primeiro flow irá executar o SNAT para o IP 5.5.5.5 se o pacote tiver como origem o IP 10.0.0.1(H1) para o destino 30.0.0.3(H3), a **action** será enviar o pacote para a **flow table 2**.

O segundo flow irá executar a ação contrária do NAT, caso o pacote esteja vindo do IP de origem 30.0.0.3(H3) com destino para o IP 5.5.5.5, será feito o NAT para o IP 10.0.0.1(H1) e a **action** será enviar o pacote para a **flow table 2**.

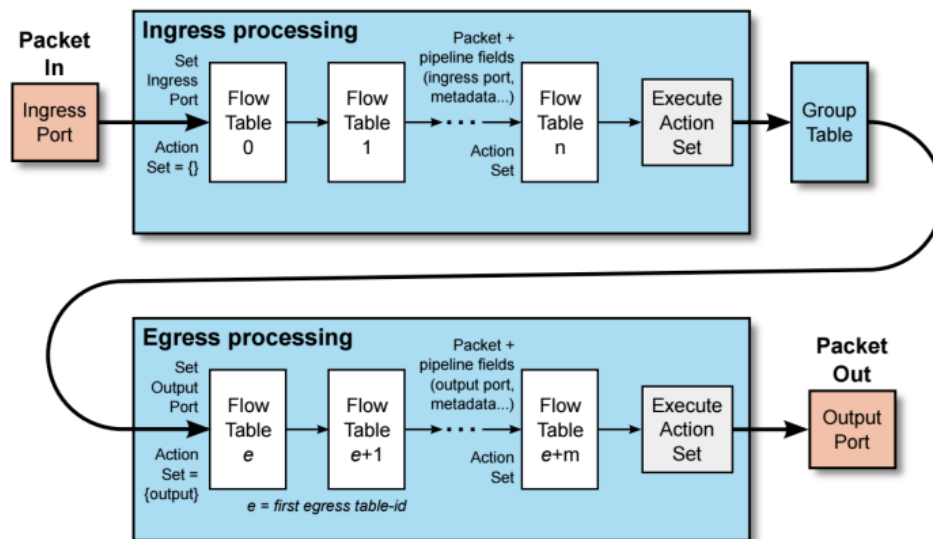
O terceiro flow irá enviar qualquer pacote que não tenha tido o **flow matching** com os flows anteriores para a **flow table 2**.

Continuando o pipeline, a **flow table 2** será responsável em encaminhar os pacotes para as portas dos IPs de destino.

Basicamente, o primeiro flow irá permitir que o tráfego de rede seja encaminhado para as portas dos hosts H1, H2 e H3 conforme as modificações realizadas na **flow table 1**.

O segundo flow, permite a resolução ARP e a comunicação entre os hosts H1 e H2 que estão na mesma subnet ocorra.

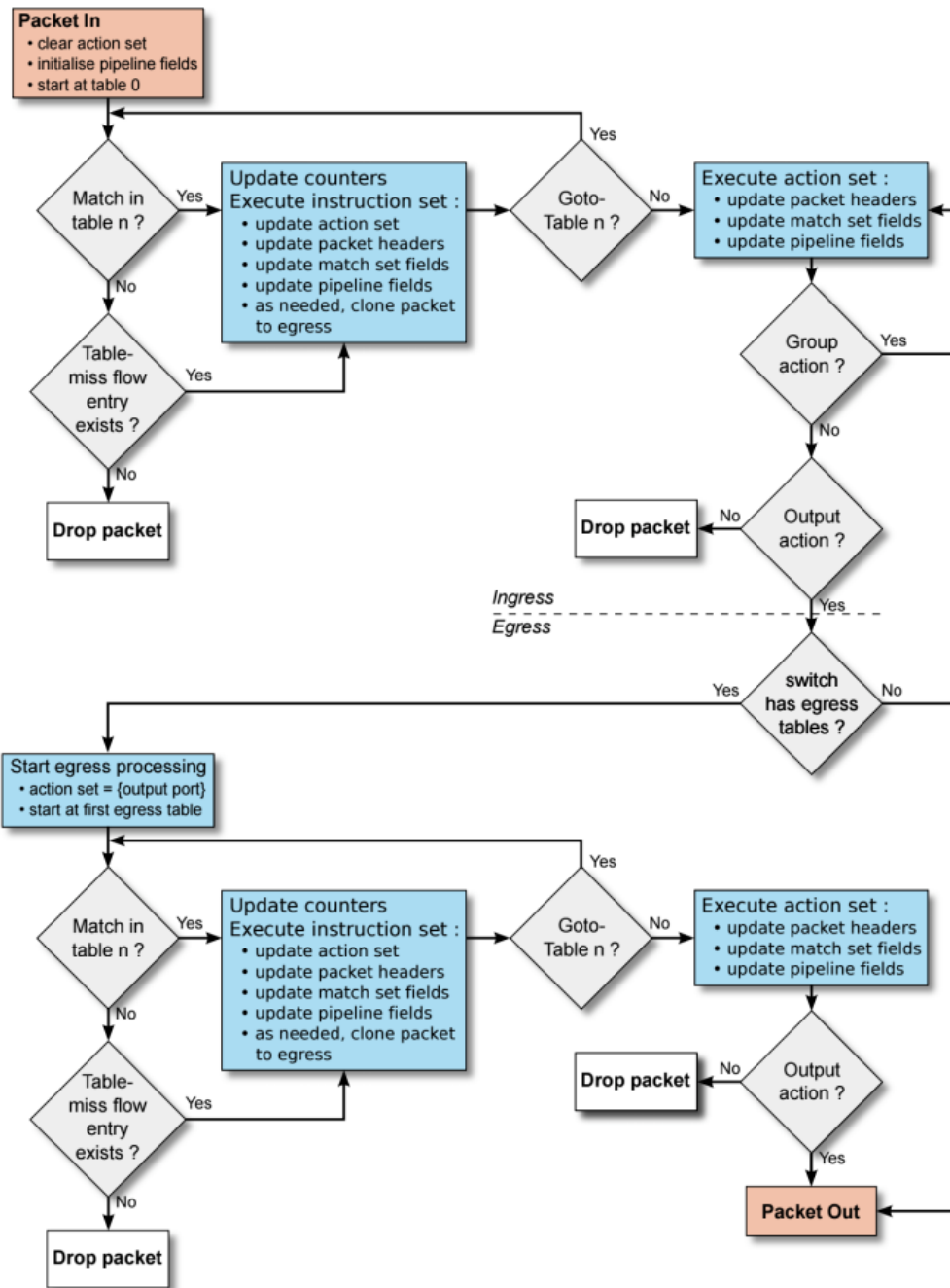
Um ponto bem importante para entender sobre o pipeline do Openflow, é que ele funciona desde o ingresso do pacote (Ingress) até a saída do pacote (Egress) conforme o diagrama abaixo:



Algo que não foi mencionado até o momento e pode ser identificado no diagrama acima é o termo **Group Table**. Resumindo, o **Group Table** é um conjunto de flow tables que realizam ações em comum, por exemplo executar o counter de pacotes para um determinado tipo de tráfego de rede ou para um caso de uso em específico como de mirror do tráfego de rede.

No treinamento 01, foi explicado o funcionamento do **flow missing** que ocorre quando o módulo de Kernel do Datapath do OVS(fast path) não possui um respectivo flow para tratar um pacote e o mesmo pergunta de forma indireta ao **ovs-vswitchd** (slow path) por um flow.

Pois bem, o **flow missing** também faz parte do pipeline do Openflow e no diagrama abaixo podemos verificar em mais detalhes todo o fluxo do pipeline:



Manipulando o OVS com o Openflow usando múltiplas flow tables [↗](#)

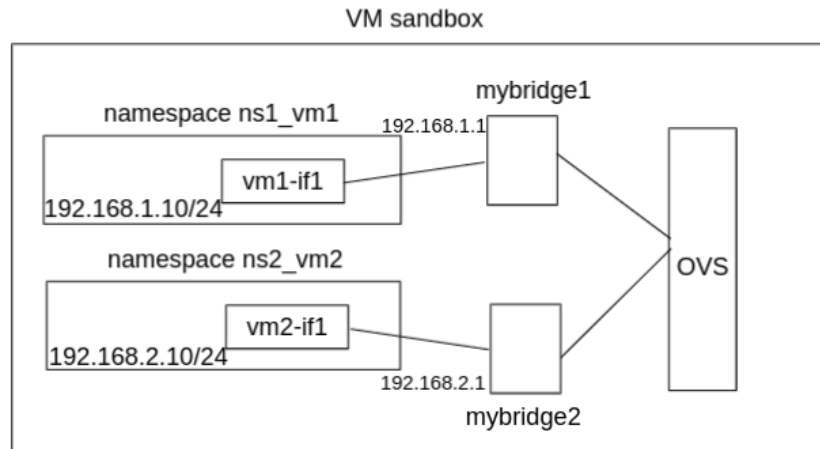
i Para fins de teste **apenas**, se recomenda ter uma VM(sandbox) executando o Ubuntu 20.04 com os pacotes latest instalados para seguir os exercícios.

1 - Instalando o OVS via pacote [↗](#)

Por favor, siga os passos do treinamento 01 para a instalação do **OVS 2.17**.

2 - Manipulando o OVS com várias flow tables do Openflow - parte 1 [↗](#)

Neste exercício, faremos a criação do setup abaixo:



Após ter finalizado a instalação do OVS, execute a seguinte sequência de comandos para criar nosso cenário:

```

1 ovs-vsctl add-br mybridge1
2 ovs-vsctl add-br mybridge2
3 ip link set dev mybridge1 up
4 ip link set dev mybridge2 up
5 ip addr add 192.168.1.1/24 dev mybridge1
6 ip addr add 192.168.2.1/24 dev mybridge2
7 ip netns add ns1_vm1
8 ip netns add ns2_vm2
9 ovs-vsctl add-port mybridge1 vm1-if1 -- set interface vm1-if1 type=internal
10 ovs-vsctl add-port mybridge2 vm2-if1 -- set interface vm2-if1 type=internal
11 ip link set vm1-if1 netns ns1_vm1
12 ip link set vm2-if1 netns ns2_vm2
13 ip netns exec ns1_vm1 ip addr add 192.168.1.10/24 dev vm1-if1
14 ip netns exec ns2_vm2 ip addr add 192.168.2.10/24 dev vm2-if1
15 ip netns exec ns1_vm1 ip link set vm1-if1 up
16 ip netns exec ns2_vm2 ip link set vm2-if1 up
17 ip netns exec ns1_vm1 ip route add 0.0.0.0/0 via 192.168.1.1
18 ip netns exec ns2_vm2 ip route add 0.0.0.0/0 via 192.168.2.1
19 echo "1" > /proc/sys/net/ipv4/ip_forward

```

i Importante: Estamos configurando o IP de gateway nas bridges de cada subnet, esse método é somente para facilitar os testes e o mesmo não é adotado em ambientes produtivos.

Nesse momento já é possível analisar se o namespace **ns1_vm1** consegue ter conectividade com o namespace **ns2_vm2** e vice-versa.

```

1 # ip netns exec ns1_vm1 ping 192.168.2.10
2 # ip netns exec ns2_vm2 ping 192.168.1.10

```

Essa comunicação só é possível pois o **ovs-vswitchd** instalou automaticamente o seguinte flow com a action NORMAL:

```

1 # ovs-ofctl dump-flows mybridge1
2 cookie=0x0, duration=1506.959s, table=0, n_packets=57, n_bytes=4322, priority=0 actions=NORMAL
3 # ovs-ofctl dump-flows mybridge2
4 cookie=0x0, duration=1508.981s, table=0, n_packets=56, n_bytes=4224, priority=0 actions=NORMAL

```

A tool **ovs-ofctl** é utilizada para manipular o **ovs-vswitchd** e faremos a remoção desse flow:

```

1 # ovs-ofctl del-flows mybridge1
2 # ovs-ofctl del-flows mybridge2

```

Verifique se existe algum flow remanescente:

```
1 # ovs-ofctl dump-flows mybridge1
2 # ovs-ofctl dump-flows mybridge2
```

Faça novamente o teste de conectividade entre os namespace **ns1_vm1** e **ns2_vm2**:

```
1 # ip netns exec ns1_vm1 ping 192.168.2.10
2 # ip netns exec ns2_vm2 ping 192.168.1.10
```

Como podem comprovar, não há conectividade entre os namespaces por conta que removemos o flow anteriormente.

3 - Manipulando o OVS com várias flow tables do Openflow - parte 2 [↗](#)

O próximo exercício é a continuação do anterior e requer que o flow padrão que existia tenha sido removido em ambas as bridges.

Faremos a implementação de flows em 3 flow tables (0, 1 e 2) para permitir o tráfego de rede entre os namespaces com IP 192.168.1.10 e 192.168.2.10 seguindo a seguinte estratégia:

- **flow table 0** - essa table é a padrão no OVS e em nosso cenário será utilizada para permitir a resolução ARP e para qualquer outro tipo de tráfego de rede IP, será encaminhado para a table 1
- **flow table 1** - nessa table faremos o roteamento entre as subnets, primeiramente por intermédio do primeiro flow como é padrão na comunicação entre subnets, o pacote terá seu MAC Address alterado para o MAC Address do gateway da subnet. E no segundo flow, permitiremos a comunicação com a subnet remota
- **flow table 2** - por último, nessa table iremos realizar o encaminhamento dos pacotes de acordo com suas portas

Primeiramente iremos adicionar os flows em ambas as bridges na **flow table 0** para permitir a resolução ARP e qualquer outro pacote IP será encaminhado para a **flow table 1**:

```
1 # ovs-ofctl add-flow mybridge1 table=0,arp,action=normal
2 # ovs-ofctl add-flow mybridge1 table=0,ip,action=goto_table=1
3 # ovs-ofctl add-flow mybridge2 table=0,arp,action=normal
4 # ovs-ofctl add-flow mybridge2 table=0,ip,action=goto_table=1
```

Antes de criarmos os flows na **flow table 1**, com a ajuda do **ovs-ofctl**, veremos quais portas/MAC Addresses estão disponíveis em cada bridge:

```
1 # ovs-ofctl show mybridge1
2 OFPT_FEATURES_REPLY (xid=0x2): dpid:00004e8fd0d41944
3 n_tables:254, n_buffers:0
4 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
5 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw_
6 1(vm1-if1): addr:6a:96:e7:e5:8c:a2
7     config:      0
8     state:       0
9     speed: 0 Mbps now, 0 Mbps max
10 LOCAL(mybridge1): addr:4e:8f:d0:d4:19:44
11     config:      0
12     state:       0
13     speed: 0 Mbps now, 0 Mbps max
14 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0
15
16 # ovs-ofctl show mybridge2
17 OFPT_FEATURES_REPLY (xid=0x2): dpid:0000a209c2cf754b
18 n_tables:254, n_buffers:0
19 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
20 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw_
21 1(vm2-if1): addr:36:85:3c:49:9b:aa
22     config:      0
23     state:       0
24     speed: 0 Mbps now, 0 Mbps max
```

```

25 LOCAL(mybridge2): addr:a2:09:c2:cf:75:4b
26     config:      0
27     state:       0
28     speed: 0 Mbps now, 0 Mbps max
29 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0

```

No output acima podemos determinar o seguinte:

- MAC Address do namespace **ns1_vm1** = 6a:96:e7:e5:8c:a2, Porta = 1
- MAC Address da bridge **mybridge1** = 4e:8f:d0:d4:19:44 , Porta = LOCAL
- MAC Address do namespace **ns2_vm2** = 36:85:3c:49:9b:aa , Porta = 1
- MAC Address da bridge **mybridge2** = a2:09:c2:cf:75:4b , Porta = LOCAL

 Faça o mesmo mapeamento com seu output

Crie os seguintes flows na **flow table 1**, que irá permitir o roteamento entre as subnets para a bridge **mybridge1** alterando os endereços de MAC Address de acordo com seu output:

```

1 # ovs-ofctl add-flow mybridge1 table=1,ip,nw_dst=192.168.1.10,action=mod_dl_src=4e:8f:d0:d4:19:44,mod_dl_dst=6a:9
2 # ovs-ofctl add-flow mybridge1 table=1,ip,nw_dst=192.168.2.0/24,action=mod_dl_src=6a:96:e7:e5:8c:a2,mod_dl_dst=4e

```

Repita o mesmo para a bridge **mybridge2**:

```

1 # ovs-ofctl add-flow mybridge2 table=1,ip,nw_dst=192.168.2.10,action=mod_dl_src=a2:09:c2:cf:75:4b,mod_dl_dst=36:8
2 # ovs-ofctl add-flow mybridge2 table=1,ip,nw_dst=192.168.1.0/24,action=mod_dl_src=36:85:3c:49:9b:aa,mod_dl_dst=a2

```

E por último, na **flow table 2** será responsável em encaminhar os pacotes para suas devidas portas em cada bridge:

```

1 # ovs-ofctl add-flow mybridge1 table=2,dl_dst=4e:8f:d0:d4:19:44,action=output:LOCAL
2 # ovs-ofctl add-flow mybridge1 table=2,dl_dst=6a:96:e7:e5:8c:a2,action=output:1
3 # ovs-ofctl add-flow mybridge2 table=2,dl_dst=a2:09:c2:cf:75:4b,action=output:LOCAL
4 # ovs-ofctl add-flow mybridge2 table=2,dl_dst=36:85:3c:49:9b:aa,action=output:1

```

 A porta **LOCAL** acima é da própria bridge.

Verifique os flows configurados em cada bridge:

```

1 # ovs-ofctl dump-flows mybridge1
2 cookie=0x0, duration=120.947s, table=0, n_packets=0, n_bytes=0, arp actions=NORMAL
3 cookie=0x0, duration=97.483s, table=0, n_packets=0, n_bytes=0, ip actions=resubmit(,1)
4 cookie=0x0, duration=75.818s, table=1, n_packets=0, n_bytes=0, ip,nw_dst=192.168.1.10 actions=mod_dl_src:4e:8f:
5 cookie=0x0, duration=67.154s, table=1, n_packets=0, n_bytes=0, ip,nw_dst=192.168.2.0/24 actions=mod_dl_src:6a:9
6 cookie=0x0, duration=40.423s, table=2, n_packets=0, n_bytes=0, dl_dst=4e:8f:d0:d4:19:44 actions=LOCAL
7 cookie=0x0, duration=29.657s, table=2, n_packets=0, n_bytes=0, dl_dst=6a:96:e7:e5:8c:a2 actions=output:"vm1-if1
8 # ovs-ofctl dump-flows mybridge2
9 cookie=0x0, duration=96.168s, table=0, n_packets=0, n_bytes=0, arp actions=NORMAL
10 cookie=0x0, duration=91.384s, table=0, n_packets=0, n_bytes=0, ip actions=resubmit(,1)
11 cookie=0x0, duration=60.585s, table=1, n_packets=0, n_bytes=0, ip,nw_dst=192.168.2.10 actions=mod_dl_src:a2:09:
12 cookie=0x0, duration=52.348s, table=1, n_packets=0, n_bytes=0, ip,nw_dst=192.168.1.0/24 actions=mod_dl_src:36:8
13 cookie=0x0, duration=22.973s, table=2, n_packets=0, n_bytes=0, dl_dst=a2:09:c2:cf:75:4b actions=LOCAL
14 cookie=0x0, duration=13.760s, table=2, n_packets=0, n_bytes=0, dl_dst=36:85:3c:49:9b:aa actions=output:"vm2-if1

```

Analisando o output acima, a entrada **actions=resubmit(1)** significa que o pacote será encaminhado para a **flow table 1** e **resubmit(2)** para a **flow table 2**.

Faça novamente o teste de conectividade entre os namespace **ns1_vm1** e **ns2_vm2**:

```

1 # ip netns exec ns1_vm1 ping 192.168.2.10

```

```
2 # ip netns exec ns2_vm2 ping 192.168.1.10
```

Como podem verificar, existe conectividade entre os namespaces.

Para analisarmos o **Datapath** e os flows existentes no fast path, use o comando abaixo porém gere algum tipo de tráfego entre os namespaces (icmp) para verificar os flows instalados:

```
1 # ovs-dpctl dump-flows
2 recirc_id(0),in_port(3),eth(src=6a:96:e7:e5:8c:a2,dst=4e:8f:d0:d4:19:44),eth_type(0x0800),ipv4(dst=192.168.2.0/25
3 recirc_id(0),in_port(2),eth(src=a2:09:c2:cf:75:4b,dst=36:85:3c:49:9b:aa),eth_type(0x0800),ipv4(dst=192.168.2.10,t
4 recirc_id(0),in_port(4),eth(src=36:85:3c:49:9b:aa,dst=a2:09:c2:cf:75:4b),eth_type(0x0800),ipv4(dst=192.168.1.0/25
5 recirc_id(0),in_port(4),eth(src=36:85:3c:49:9b:aa,dst=a2:09:c2:cf:75:4b),eth_type(0x0806), packets:1, bytes:42, u
6 recirc_id(0),in_port(2),eth(src=a2:09:c2:cf:75:4b,dst=36:85:3c:49:9b:aa),eth_type(0x0806), packets:0, bytes:0, us
7 recirc_id(0),in_port(3),eth(src=6a:96:e7:e5:8c:a2,dst=4e:8f:d0:d4:19:44),eth_type(0x0806), packets:0, bytes:0, us
8 recirc_id(0),in_port(1),eth(src=4e:8f:d0:d4:19:44,dst=6a:96:e7:e5:8c:a2),eth_type(0x0800),ipv4(dst=192.168.1.10,t
9 recirc_id(0),in_port(1),eth(src=4e:8f:d0:d4:19:44,dst=6a:96:e7:e5:8c:a2),eth_type(0x0806), packets:1, bytes:42, u
```

No output acima existem dois tipos de fluxo, o primeiro é a resolução **ARP** identificado pelo **eth_type(0x0806)** e a comunicação de **IP** com **IP** identificado por **eth_type(0x0800)**.

4 - Manipulando o OVS com várias flow tables do Openflow - parte 3 [↗](#)

Nesse exercício, faremos o trace dos pacotes conforme eles atravessam o **OVS**. O tool **ovs-appctl** dá uma visão do que acontece com os pacotes assim que eles são processados pelo **Datapath** (modificação do header do pacote, porta de output e etc).

Para nosso exemplo, faremos uma análise no seguinte fluxo de comunicação:

Protocolo de rede: IP

IP de origem: 192.168.1.10

IP de destino: 192.168.2.10

MAC Address de origem: 6a:96:e7:e5:8c:a2 (altere o mac address)

TTL: 64

Execute o trace abaixo na bridge **mybridge1** utilizando os valores acima:

```
1 # ovs-appctl ofproto/trace mybridge1 ip,nw_src=192.168.1.10,nw_dst=192.168.2.10,dl_src=6a:96:e7:e5:8c:a2,nw_ttl=
2 Flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_ds
3
4 bridge("mybridge1")
5 -----
6 0. ip, priority 32768
7   resubmit(,1)
8 1. ip,nw_dst=192.168.2.0/24, priority 32768
9   mod_dl_src:6a:96:e7:e5:8c:a2
10  mod_dl_dst:4e:8f:d0:d4:19:44
11  dec_ttl
12  resubmit(,2)
13 2. dl_dst=4e:8f:d0:d4:19:44, priority 32768
14   LOCAL
15
16 Final flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=6a:96:e7:e5:8c:a2,dl_dst=4e:8f:d0:d4:19:44,nw_src=192.168.1.10
17 MegafLOW: recirc_id=0,eth,ip,in_port=ANY,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_dst=192.168.2.0/24
18 Datapath actions: set(eth(dst=4e:8f:d0:d4:19:44)),1
```

O output acima se divide em três partes:

Primeira parte:

```
1 Flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.2.0/24
```

Os valores dão match com o que foi passado no comando de trace. Para alguns valores, o trace irá usar o padrão se não especificado.

Segunda parte:

```
1 bridge("mybridge1")
2 -----
3 0. ip, priority 32768
4   resubmit(,1)
5 1. ip,nw_dst=192.168.2.0/24, priority 32768
6   mod_dl_src:6a:96:e7:e5:8c:a2
7   mod_dl_dst:4e:8f:d0:d4:19:44
8   dec_ttl
9   resubmit(,2)
10 2. dl_dst=4e:8f:d0:d4:19:44, priority 32768
11   LOCAL
```

Mostra os flows que deram match nas **flows tables**. Acima, você pode verificar que o pacote entrou na **flow table 0** e foi resubmitted para a **flow table 1**. Na **flow table 1**, o frame do pacote será modificado com o novo MAC Address e resubmitted para a **flow table 2**. Ao chegar na **flow table 2**, o pacote irá sair pela porta LOCAL que é correspondente a bridge **mybridge1**.

Terceira parte:

```
1 Final flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=6a:96:e7:e5:8c:a2,dl_dst=4e:8f:d0:d4:19:44,nw_src=192.168.1.10,nw_dst=192.168.2.0/24,
2 MegafLOW: recirc_id=0,eth,ip,in_port=ANY,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.2.0/24,
3 Datapath actions: set(eth(dst=4e:8f:d0:d4:19:44)),1
```

Mostra os campos dos flows após ter sido processado pelo **Datapath**.

Para entender o final da comunicação, execute o trace com as mesmas opções para a bridge **mybridge2**:

```
1 # ovs-appctl ofproto/trace mybridge2 ip,nw_src=192.168.1.10,nw_dst=192.168.2.10,dl_src=6a:96:e7:e5:8c:a2,nw_ttl=64
2 Flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.2.10
3
4 bridge("mybridge2")
5 -----
6 0. ip, priority 32768
7   resubmit(,1)
8 1. ip,nw_dst=192.168.2.10, priority 32768
9   mod_dl_src:a2:09:c2:cf:75:4b
10  mod_dl_dst:36:85:3c:49:9b:aa
11  dec_ttl
12  resubmit(,2)
13 2. dl_dst=36:85:3c:49:9b:aa, priority 32768
14   output:1
15
16 Final flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=a2:09:c2:cf:75:4b,dl_dst=36:85:3c:49:9b:aa,nw_src=192.168.1.10,nw_dst=192.168.2.10,nw_ttl=64
17 MegafLOW: recirc_id=0,eth,ip,in_port=ANY,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.2.10,nw_ttl=64
18 Datapath actions: set(eth(src=a2:09:c2:cf:75:4b,dst=36:85:3c:49:9b:aa)),4
```

5 - Manipulando o OVS com várias flow tables do Openflow - parte 4 [↗](#)

Nesse exercício, continuaremos a estudar o trace dos pacotes conforme eles atravessam o **OVS**.

Avalie o fluxo de comunicação entre o IP **192.168.1.10** e seu gateway **192.168.1.1**:

```
1 # ip netns exec ns1_vm1 ping 192.168.1.1
2 PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data:
3 ^C
```

```
4 --- 192.168.1.1 ping statistics ---
5 2 packets transmitted, 0 received, 100% packet loss, time 1015ms
```

Avaliando os flows instalados na bridge **mybridge1**, podemos verificar que não existe nenhum flow correspondente que permita essa comunicação:

```
1 # ovs-ofctl dump-flows mybridge1
2 cookie=0x0, duration=19098.707s, table=0, n_packets=16, n_bytes=672, arp actions=NORMAL
3 cookie=0x0, duration=19075.243s, table=0, n_packets=26, n_bytes=2548, ip actions=resubmit(,1)
4 cookie=0x0, duration=19053.578s, table=1, n_packets=12, n_bytes=1176, ip,nw_dst=192.168.1.10 actions=mod_dl_src:
5 cookie=0x0, duration=19044.914s, table=1, n_packets=12, n_bytes=1176, ip,nw_dst=192.168.2.0/24 actions=mod_dl_sr
6 cookie=0x0, duration=19018.183s, table=2, n_packets=12, n_bytes=1176, dl_dst=4e:8f:d0:d4:19:44 actions=LOCAL
7 cookie=0x0, duration=19007.417s, table=2, n_packets=12, n_bytes=1176, dl_dst=6a:96:e7:e5:8c:a2 actions=output:"v
```

É possível verificar que ao menos a resolução **ARP** funcionou, já que temos um flow permitindo esse tipo de tráfego de rede:

```
1 # ip netns exec ns1_vm1 arp -a -n
2 ? (192.168.1.1) at 4e:8f:d0:d4:19:44 [ether] on vm1-if1
```

Execute o trace do pacote simulando esse fluxo de comunicação(altere o mac address conforme seu setup):

```
1 # ovs-appctl ofproto/trace mybridge1 ip,nw_src=192.168.1.10,nw_dst=192.168.1.1,dl_src=6a:96:e7:e5:8c:a2,nw_ttl=6
2 Flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=6a:96:e7:e5:8c:a2,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_ds
3
4 bridge("mybridge1")
5 -----
6 0. ip, priority 32768
7     resubmit(,1)
8 1. No match.
9     drop
10
11 Final flow: unchanged
12 MegafLOW: recirc_id=0,eth,ip,in_port=ANY,nw_dst=192.168.1.0/29,nw_frag=no
13 Datapath actions: drop
```

Como é possível verificar, o pacote entrou na **flow table 0** e foi resubmitted para a **flow table 1** e na mesma não houve nenhum **flow matching** e a **action** final foi **drop**.

Para resolver isso, vamos criar o flow correspondente na **flow table 1** para permitir esse tráfego de rede:

```
1 # ovs-ofctl add-flow mybridge1 table=1,ip,nw_dst=192.168.1.1,nw_src=192.168.1.10,action=goto_table=2
```

Verifique os flows criados:

```
1 # ovs-ofctl dump-flows mybridge1
2 cookie=0x0, duration=20281.114s, table=0, n_packets=24, n_bytes=1008, arp actions=NORMAL
3 cookie=0x0, duration=20257.650s, table=0, n_packets=33, n_bytes=3231, ip actions=resubmit(,1)
4 cookie=0x0, duration=20235.985s, table=1, n_packets=14, n_bytes=1372, ip,nw_dst=192.168.1.10 actions=mod_dl_src:
5 cookie=0x0, duration=20227.321s, table=1, n_packets=12, n_bytes=1176, ip,nw_dst=192.168.2.0/24 actions=mod_dl_sr
6 cookie=0x0, duration=40.398s, table=1, n_packets=2, n_bytes=196, ip,nw_src=192.168.1.10,nw_dst=192.168.1.1 actio
7 cookie=0x0, duration=20200.590s, table=2, n_packets=14, n_bytes=1372, dl_dst=4e:8f:d0:d4:19:44 actions=LOCAL
8 cookie=0x0, duration=20189.824s, table=2, n_packets=14, n_bytes=1372, dl_dst=6a:96:e7:e5:8c:a2 actions=output:"v
```

No output temos o seguinte flow:

```
cookie=0x0, duration=40.398s, table=1, n_packets=2, n_bytes=196, ip,nw_src=192.168.1.10,nw_dst=192.168.1.1
actions=resubmit(,2) .
```

Execute o trace do pacote simulando esse fluxo de comunicação(altere o mac address conforme seu setup):

```

1 # ovs-appctl ofproto/trace mybridge1 ip,nw_src=192.168.1.10,nw_dst=192.168.1.1,nw_ttl=64
2 Flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=00:00:00:00:00:00,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.1.1,nw_ttl=64
3
4 bridge("mybridge1")
5 -----
6 0. ip, priority 32768
7     resubmit(,1)
8 1. ip,nw_src=192.168.1.10,nw_dst=192.168.1.1, priority 32768
9     resubmit(,2)
10 2. No match.
11     drop
12
13 Final flow: unchanged
14 MegafLOW: recirc_id=0,eth,ip,in_port=ANY,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.1.1,nw_ttl=64
15 Datapath actions: drop

```

Ainda temos um drop, isso é devido a **flow table 2** não encontrar nenhum flow referente a IP, vamos criá-lo conforme abaixo:

```

1 # ovs-ofctl add-flow mybridge1 table=2,ip,nw_dst=192.168.1.1,action=output:LOCAL

```

Refaça o trace novamente:

```

1 # ovs-appctl ofproto/trace mybridge1 ip,nw_src=192.168.1.10,nw_dst=192.168.1.1,nw_ttl=64
2 Flow: ip,in_port=ANY,vlan_tci=0x0000,dl_src=00:00:00:00:00:00,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.1.1,nw_ttl=64
3
4 bridge("mybridge1")
5 -----
6 0. ip, priority 32768
7     resubmit(,1)
8 1. ip,nw_src=192.168.1.10,nw_dst=192.168.1.1, priority 32768
9     resubmit(,2)
10 2. ip,nw_dst=192.168.1.1, priority 32768
11     LOCAL
12
13 Final flow: unchanged
14 MegafLOW: recirc_id=0,eth,ip,in_port=ANY,dl_dst=00:00:00:00:00:00,nw_src=192.168.1.10,nw_dst=192.168.1.1,nw_ttl=64
15 Datapath actions: 1

```

Com o output acima é possível determinar que existe um flow correspondente ao fluxo que estamos testando.

Avalie o fluxo de comunicação entre o IP **192.168.1.10** e seu gateway **192.168.1.1** novamente:

```

1 # ip netns exec ns1_vm1 ping 192.168.1.1
2 PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
3 64 bytes from 192.168.1.1: icmp_seq=1 ttl=63 time=0.240 ms
4 64 bytes from 192.168.1.1: icmp_seq=2 ttl=63 time=0.073 ms
5 ^C
6 --- 192.168.1.1 ping statistics ---
7 2 packets transmitted, 2 received, 0% packet loss, time 1012ms
8 rtt min/avg/max/mdev = 0.073/0.156/0.240/0.083 ms

```

Desafio

Após ter aprendido como podemos manipular OVS com os flows em diferentes tables do Openflow, tente resolver o seguinte desafio:

1 - Faça a criação de 4 namespaces conectados e duas bridges gerenciadas pelo OVS

Sugestão: mybridge1: ns1_vm1, ns2_vm2 e mybridge2: ns3_vm3, ns4_vm4

2 - Crie duas subnets e atribua os endereços de gateway para as bridges e configure os namespaces ns1_vm1, ns2_vm2 para a subnet 1 e ns3_vm3 e ns4_vm4 na subnet2

3 - Utilizando o conceito de flow tables, permita o tráfego de rede somente entre ns1_vm1 e ns3_vm3 e ns2_vm2 com ns4_vm4 e crie os flows conforme abaixo:

- na flow table 0 libere o tráfego arp local e envie o tráfego de rede para a flow table 1
- ao chegar na flow table 1, permita o roteamento entre as subnets e envie o tráfego para o flow table 2
- na flow table 2, permita o encaminhamento do tráfego de rede para as portas de destino

Conclusão

Neste treinamento foi possível entender a manipulação do OVS via Openflow com flows em diferentes flow tables.

Abaixo existe o passo a passo para realizar o cleanup na VM utilizada para os exercícios.

```
1 # apt remove -y openvswitch-switch openvswitch-common python3-openvswitch
2 # rm -rf /var/lib/openvswitch
3 # rm -rf /var/log/openvswitch
4 # rm -rf /etc/openvswitch
5 # rm -f /etc/default/openvswitch-switch
6 # ip netns delete ns1_vm1
7 # ip netns delete ns2_vm2
8 # reboot
```